

Świat nie kończy się na ekranie

Świat, kamera i viewport

Świat gry jest większy od ekranu. Kamera wybiera jego fragment. Viewport wyświetla to, co widzi kamera.



- 1 Świat gry jest duży.**
Zawiera całą mapę, wszystkie obiekty i elementy gry.
- 2 Kamera wybiera fragment świata.**
Kamera to prostokątne okno, które określa, co aktualnie widzimy.
- 3 Viewport wyświetla obraz z kamery.**
Na ekranie widoczny jest tylko ten fragment, który znajduje się w kamerze, powiększony do rozmiaru ekranu.

Kamera decyduje, co widzimy. UI jest przypięte do ekranu, a nie do mapy.

Świat przechowuje pozycje obiektów. **Kamera** ustala, który fragment świata widzimy. **Viewport** to obszar wyświetlania obrazu gry - w naszym kursie jest nim canvas, nie całe okno przeglądarki.

Ilustracja pokazuje ogólną ideę. W obecnym GameLab kamera przesuwa widok, ale nie ma zoomu. Większy rozmiar canvasu może pokazać większy fragment świata; skalowanie canvasu przez CSS zmienia rozmiar obrazu na stronie, nie fizyczne pozycje obiektów.

UI / HUD to interfejs gracza: punkty, zdrowie, czas. Pozostaje przy ekranie, nawet gdy kamera przesuwa mapę.

Pozycja świata a pozycja ekranu

Bez zoomu i drżenia: **ekran = świat - przesunięcie widoku**. Pozycja obiektu w świecie nie zmienia się tylko dlatego, że przesuwasz kamerę.

```
screenX = worldX - viewX;  
screenY = worldY - viewY;
```

Przykład: gracz jest w świecie w (500, 300), a lewy górny róg widoku w (180, 120). Na ekranie gracz jest w (320, 180). Skrzynka w (700, 300) pojawi się na ekranie w (520, 180).

Śledzenie gracza w GameLab

`follow(player)` oblicza przesunięcie tak, aby gracz był w środku canvasu. Dla canvasu 640 × 360 środek to (320, 180). Oto fragment do `onCreate()`, po utworzeniu gracza:

```
const cameraObject = this.createObject('Camera');  
const camera = new GameLab.Camera2D();  
cameraObject.addComponent(camera);  
camera.follow(player);  
  
// Pozwól graczowi wyjść poza początkowy canvas:  
const controller = player.getComponent(  
  GameLab.CharacterController2D  
);  
controller.collideWorldBounds = false;
```

Domyślna granica kontrolera to obszar canvasu, nie wielkość Twojej mapy. Wyłączenie jej pozwala badać większy świat. Własne ściany i granice dodasz osobno. Sam obiekt Camera i jego `setPosition` nie sterują kamerą: używaj `follow` lub `offsetX/Y`.

Przesuwaj widok, nie obiekty

```
camera.stopFollowing();
camera.offsetX = 180;
camera.offsetY = 120;

// Ponownie śledź gracza bez dodatkowego offsetu:
camera.offsetX = 0;
camera.offsetY = 0;
camera.follow(player);

// Krótkie drżenie obrazu, bez ruchu gracza:
camera.shake(4, 0.2);
```

Gdy kamera śledzi obiekt, offset dodaje się do obliczonego przesunięcia. Dodatni offsetX przesuwa lewy brzeg widoku w prawo, więc świat wygląda, jakby przesunął się w lewo. Po stopFollowing() offset jest przesunięciem względem początku świata.

Tło i obiekty

```
const ground = this.createObject('Ground');
ground.addComponent(new GameLab.TileMap('grass', 32));

const marker = this.createObject('Marker');
marker.setPosition(700, 300);
marker.addComponent(new GameLab.Sprite('chest', 32, 32));
```

TileMap rysuje powtarzalne tło, które przesuwa się z kamerą. Nie tworzy kolizji ani ograniczonej mapy. Do obserwowania ruchu kamery dodaj kilka markerów o różnych pozycjach świata.

Ćwiczenie

Ustaw Marker w (700, 300). Gracz jest w (500, 300), canvas ma 640 × 360. Przewidź pozycję ekranową Markera przy follow(player), potem uruchom grę. Zmień offsetX na 40 i porównaj wynik.

Tekst przypięty do ekranu

UI dodaj jako komponent TextRenderer na osobnym obiekcie HUD. Jego x i y są współrzędnymi ekranu. Kamera ich nie przesuwają. Y wskazuje linię bazową tekstu, a nie jego górną krawędź.

```
// Fragment do onCreate() sceny:
this.points = 0;
const hud = this.createObject('HUD');
const score = new GameLab.TextRenderer(
  () => `Punkty: ${this.points}`,
  12, 24, '#ffffff'
);
hud.addComponent(score);
hud.addComponent(new GameLab.ControlsHint(
  'Kliknij planszę. WASD: ruch'
));
```

Funkcja przekazana do TextRenderer odczytuje aktualny stan przy rysowaniu. Nie dodawaj nowego TextRenderer w każdej klatce. Aby przetestować aktualizację, zmień this.points w istniejącym onUpdate sceny:

```
if (this.input.isKeyPressed(' ')) {
  this.points += 1;
}
```

isKeyPressed reaguje na nowe naciśnięcie; isKeyDown oznacza przytrzymanie i naliczałby punkty wiele razy. ControlsHint pokazuje instrukcję, kiedy plansza nie ma fokusu.

Wybór UI: TextRenderer nadaje się do tekstowego HUD-u w grze. Menu z przyciskami, formularzami i rozbudowanym układem możesz zbudować w HTML/CSS obok canvasu. Nie traktuj zwykłego sprite'a świata jako tekstu HUD-u.

Mapa się przesuwa, HUD zostaje

Rozbuduj postać WASD z poprzedniego kursu. Dodaj Camera2D śledzącą gracza, wyłącz ograniczenie kontrolera do canvasu, dodaj trawę i trzy skrzynki: (100, 100), (700, 300), (900, 600).

Dodaj HUD z punktami oraz instrukcją sterowania. Spacja zwiększa punkty o jeden na nowe naciśnięcie. Idź do odległych skrzynek i sprawdź, że HUD nadal znajduje się w tym samym miejscu.

Sprawdź rozwiązanie

1. Gracz pozostaje w centrum widoku przy offsetach równych 0.
2. Markery i tło przesuwały się na ekranie, a ich pozycje świata się nie zmieniają.
3. HUD nie podąża za kamerą.
4. Przytrzymanie spacji nie dodaje punktów co klatkę.
5. Ruch poza pierwotny canvas nie jest blokowany jego granicą.

Pytania i odpowiedzi

Czy obiekt poza ekranem znika ze świata? **Nie.**

Czy kamera zmienia kolizje? **Nie, fizyka używa świata.**

Marker przy follow gracza z przykładu: **(520, 180).**

Po dodaniu offsetX = 40: **(480, 180).**

Czy ten silnik ma zoom kamery? **Nie w obecnym API.**

Kod odpowiada lokalnemu GameLab. Rysunek jest schematem idei kamery, a nie obietnicą gotowej mapy lub systemu zoomu.